

Efficient Memory Virtualization for Cross-ISA System Mode Emulation

Chao-Jui Chang, Jan-Jan Wu
Institute of Information Science,
Academia Sinica, Taiwan
{crchang,wuj}@iis.sinica.edu.tw

Wei-Chung Hsu, Pangfeng Liu
Dept. Computer Science &
Information Engineering
National Taiwan University
{hsuwc, pangfeng}@csie.ntu.edu.tw

Pen-Chung Yew
Dept. Computer Science
& Engineering
University of Minnesota at Twin
Cities, Minneapolis, USA
yew@cs.umn.edu

Abstract

Cross-ISA system-mode emulation has many important applications. For example, Cross-ISA system-mode emulation helps computer architects and OS developers trace and debug kernel execution-flow efficiently by emulating a slower platform (such as ARM) on a more powerful platform (such as an x86 machine). Cross-ISA system-mode emulation also enables workload consolidation in data centers with platforms of different instruction-set architectures (ISAs). However, system-mode emulation is much slower. One major overhead in system-mode emulation is the multi-level memory address translation that maps guest virtual address to host physical address. Shadow page tables (SPT) have been used to reduce such overheads, but primarily for same-ISA virtualization. In this paper we propose a novel approach called embedded shadow page tables (ESPT). EPST embeds a shadow page table into the address space of a cross-ISA dynamic binary translation (DBT) and uses hardware memory management unit in the CPU to translate memory addresses, instead of software translation in a current DBT emulator like QEMU. We also use the larger address space on modern 64-bit CPUs to accommodate our DBT emulator so that it will not interfere with the guest operating system. We incorporate our new scheme into QEMU, a popular, retargetable cross-ISA system emulator. SPEC CINT2006 benchmark results indicate that our technique achieves an average speedup of 1.51 times in system mode when emulating ARM on x86, and a 1.59 times speedup for emulating IA32 on x86_64.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
VEE '14, March 1–2, 2014, Salt Lake City, Utah, USA.
Copyright © 2014 ACM 978-1-4503-2764-0 /14/03...\$15.00.
<http://dx.doi.org/10.1145/2576195.2576201>

Categories and Subject Descriptors D.0 [Computer Systems Organization]: System Architectures;

General Terms Design, Experimentation, Performance

Keywords memory virtualization; cross-ISA dynamic binary translation; hardware MMU; embedded shadow page table

1. Introduction

Whole system emulation is very important for multi-cores and cloud computing. It allows applications running on such systems to be agnostic about the underlying operating systems and hardware platforms. For example, applications compiled for a particular instruction-set architecture (ISA) and operating system, such as an Intel x86 platform running Windows, can run on an ARM platform running Linux. With system mode emulation we can run legacy applications developed for older OSes and ISAs on newer ones. System mode emulation also enables the implementation of secure environments in which operating systems are isolated from one another. Finally system emulation speeds up CPU execution flow tracing and OS kernel debugging by emulating a slower platform (e.g., ARM) on a faster one (e.g., x86-64).

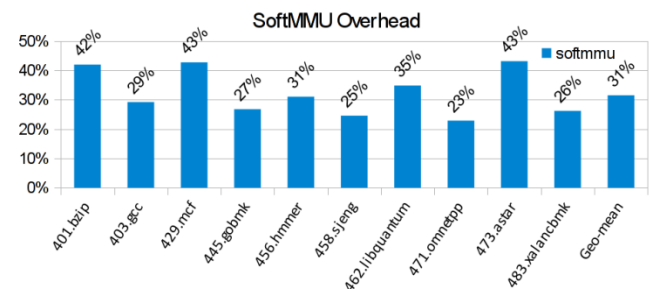


Figure 1-0. Percentage of the total execution time spent in address translation in QEMU system-mode emulation (ARM to x86_64)

For same ISA system emulation, efficiency is improved by recent hardware support, including Intel VT-x, AMD-V, and ARM-VE. This paper focuses on cross-ISA system-level emulation, i.e., the guest and the host are of different instruction set architectures, using dynamic binary translation (DBT) techniques.

QEMU is a popular retargetable cross-ISA, system-level emulator. It runs as a process and virtualizes memory through multi-level address translation by the software-based virtual memory management unit (vMMU). The address translation incurs significant overheads and is the main reason for slow emulation. For example, Figure 1-0 shows that in average QEMU spends 23%~43% of the total execution time in memory translation for SPEC CINT2006 benchmarks when running in system mode emulation.

In this paper, we propose an efficient memory virtualization scheme for cross-ISA system-level DBT. This scheme effectively incorporates shadow page tables (SPTs) [20] in DBT and uses the hardware MMU on the host machine to translate memory addresses without slow software MMU translation.

Using Shadow Page Tables (SPTs) or hardware assisted Second Level Address Translation (SLAT) [24,26] to support memory virtualization is a well-known technique for same-ISA virtualization [17]. However, adopting a shadow page table in a cross-ISA DBT is much more challenging for the three reasons. First, the host shadow page table must be carefully placed to avoid interfering with the virtual memory space of the guest OS, because we are collapsing the two address space. Second, the overheads in creating and maintaining a shadow page table must be minimized, because guest page tables modification is just a normal memory operation and tracing all guest memory update incurs extra overhead. Third, multi-process emulation with one address space, because multiple processes are executed at the same time in a guest OS.

We propose the notion of “*embedded shadow page table*” (ESPTs) to address these challenges. ESPT utilizes the larger address space of modern 64-bit CPUs to provide more space for mapping tables and to achieve address isolation. The rationale behind the embedded shadow page table is that by embedding the shadow page entries into the host page table of the DBT, the translated host binary code will be executed in the address space of the host machine, and thus avoiding switching between shadow and page tables while translating a guest virtual address.

The main contributions of this paper are as follows.

- We propose an efficient memory virtualization for cross-ISA system-level DBT by incorporating hardware MMU into the DBT. To the best of our knowledge, our work is the first effort to exploit hardware MMU support for cross-ISA, system-level memory virtualization.
- We propose the notion of embedded shadow page tables to avoid frequent switching between host page table and shadow page table during address translation.
- We adopt an interception-based methodology for tracing guest page modification, reducing the overhead of synchronizing guest page tables and embedded shadow page tables.
- Our implementation on QEMU demonstrates an average speedup of 1.51 times and 1.59 times on SPEC CINT2006 benchmarks for ARM-to-x86 and IA32-to-x86_64 system-mode emulation, respectively, when compared with the standard soft-MMU approach in QEMU. The results also show an average speedup of 1.15 times and 1.28 times on NBench benchmarks for ARM-to-x86 and IA32-to-x86_64 system-mode emulation, respectively.

The rest of the paper is organized as follows. Section 2 gives an overview on the various memory virtualization techniques and the motivation of our proposed ESPT-based approach for cross-ISA memory address translation. We also elaborate on the challenges in the design and implementation of such an approach. Section 3 presents our design and implementation details of the proposed ESPT-based approach. Section 4 reports our experiment results on ARM-to-x86 emulation and x86_32 to x86_64 emulation with SPEC CINT2006 and the NBench benchmarks. Section 5 describes some related work, and Section 6 gives our concluding remarks.

2. Memory Virtualization

In this section, we first briefly describe memory address translation and DBT emulation. Next, we introduce software-based same-ISA and cross-ISA emulation, and point out the source of inefficiency in such software-based implementation. We then introduce recent hardware approaches for same-ISA memory address translation, e.g., Intel's Extended Page Table (EPT) and AMD's Nested Paging, and explain why they cannot provide efficient support for cross-ISA emulation. Finally we describe the motivation behind our hardware-assisted approach for cross-ISA memory virtualization, and discuss the challenges in designing and implementing the proposed approach.

Virtual address technology provides a separate address space for each process to achieve process isolation in a multitasking operating system with the help of a memory management unit (MMU). MMU is a computer hardware component that provides memory access for CPU by translating virtual addresses from processes to physical addresses. A physical address is the memory address on a physical machine.

Figure 2-0 illustrates the components in a DBT emulation system and the execution flow. Virtualization technol-

ogy allows multiple operating systems to run on a single physical machine. The operating system runs directly on the physical machine is the host operating system, and the operating systems running on the host operating system are guest operating systems. A DBT emulator is a process that runs on the host operating system and provides emulated hardware components, like a virtual MMU, CPU, and memory, to the guest operating systems.

The execution flow of a DBT is: 1) a user launches a DBT emulator to run a guest OS on host operating system; 2) an application in the guest OS issues a memory access instruction; 3-4) the software MMU refers to the guest's page table and performs the translation of virtual address (GVA) to physical address (GPA); 5-6) the DBT emulator has another layer of mapping guest physical address to host virtual address (HVA) to avoid occupying the whole host memory at beginning; 7-8) the host memory management unit refers to host's page table and performs host virtual address to host physical address (HPA) translation. Note that in a cross-ISA DBT, the guest physical address is not the same as the host virtual address, whereas in a same-ISA VM they are basically equivalent.

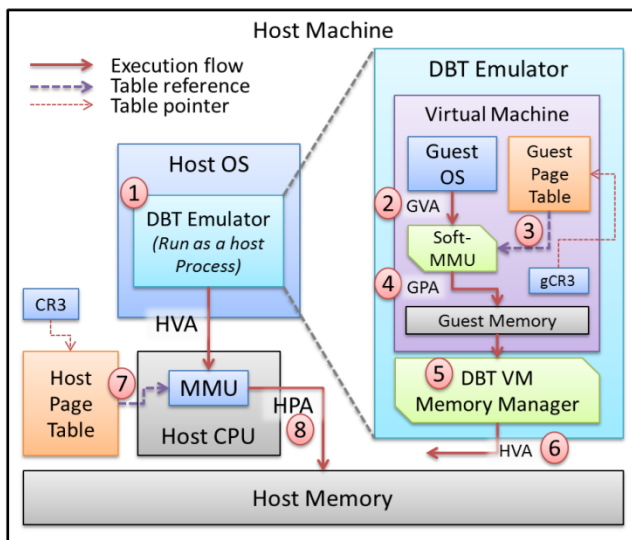


Figure 2-0. The components and the execution flow of a DBT emulator

For same-ISA virtualization, modern CPU provides hardware support to execute the guest OS seamlessly while use guest page table directly. When address translation is needed, the host CPU will first look up the guest page table and then host page table to get host physical address.

For cross-ISA emulation, the host cannot run a guest OS directly because the binary of the guest OS is not compatible with the host CPU. To execute the guest OS, we need a dynamic binary translator (DBT) to translate the guest binary into a host binary, then we run the host binary on host CPU. If the DBT needs to translate a memory address,

it needs to go through step 3 to 6 in Figure 2-0 to find the corresponding host virtual address (HVA). Then, the DBT emulator accesses the translated address just like a normal host user process accesses a memory address, and return the result to the guest OS. The DBT emulator must complete this multi-step translation for every memory access.

2.1 Design Goal for Memory Address Translation in Cross-ISA Emulation

A cross-ISA system-mode DBT needs to go through three levels of address translation for each guest memory access, while emulating a guest VM, as shown Figure 2-1-1(a). Only the last level of address translation is performed by hardware MMU on the host machine, and the first two levels of address translation between guest OS and host are implemented purely in software. This software-implemented address translation subsystem is referred to as soft-MMU, as shown within the circle of Figure 2-1-1(a). The soft-MMU approach is one of the biggest performance bottlenecks in cross-ISA system-mode DBT, such as QEMU.

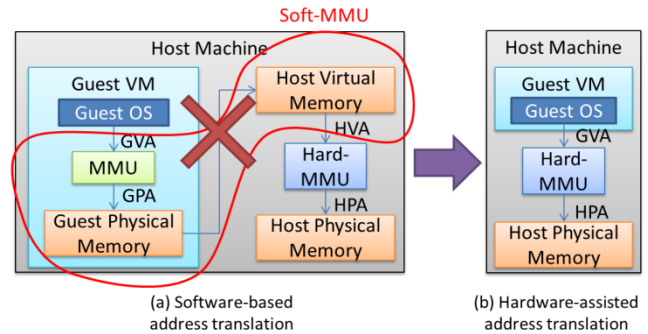


Figure 2-1-1. Address translation for cross-ISA emulation

We improve address translation performance by using hardware MMU to translate virtual address on virtual machine directly to physical address on physical machine without soft-MMU, as shown in Figure 2-1-1(b). We also take advantage of a larger address space available on modern 64-bit CPUs to isolate the DBT emulator from the guest operating system, as shown in Figure 2-1-2.

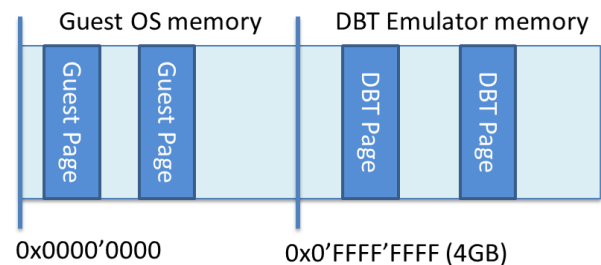


Figure 2-1-2. Memory layout with wider memory space on 64-bit CPUS

In Section 2.2 and 2.3, we introduce two memory address translation techniques currently used for same-ISA memory virtualization: (1) software based shadow page table and (2) hardware assisted second level address translation. We then point out the reasons why these same-ISA techniques are not sufficient for cross-ISA memory address translation.

2.2 Same-ISA Address Translation

Shadow Page Table

For security reasons, guest processes are not allowed to access page tables on hardware without page table virtualization. Consequently, in a pure software implementation, the host Virtual Machine Monitor (VMM) creates a shadow page table (SPT) that maps guest address to host address for each guest page table and changes the page table base pointer (CR3) to the shadow page table. Then the guest process can use the shadow page table as if it were using its own guest page table for address translation.

Shadow page table is an effective technique in same-ISA address translation, but not in cross-ISA translation. In same-ISA virtualization, host CPU can execute guest OS binary directly, thus VMM only needs to switch between shadow page tables when the guest OS switches context. However, in cross-ISA, the guest OS has been translated into host binary and each guest memory access also been translated into Soft-MMU operations. If we adopt SPT technology into cross-ISA, frequent page table switching between host and shadow page table will cause huge performance degradation. The reason is that the translated guest binary and the DBT emulator now use the same host address space, so the DBT emulator must switch to shadow page table when it needs to translate addresses for the guest OS, and then switch back to the host page table of the DBT emulator to run the translated guest binary. The DBT engine needs to protect from being modified by guest OS. In same-ISA, Bugnion et al. propose a segmentation-based solution[19], however it only works on x86 machine. To achieve better portability, we check the translated host virtual address and make sure it doesn't touch the DBT engine before embedding into the host page table.

Second Level Address Translation

Intel Extended Page Tables (EPT) [24,25] and AMD Nested Paging [26] are x86 virtualization technologies to avoid overheads in maintaining software-based shadow page tables. When the host machine runs a guest process, it will first load the guest page table to translate guest address, and then load the EPT table to translate guest address to host address. In contrast with aforementioned software-based shadow page table approach, the EPT technology uses the guest page table directly and thus does not require the host virtual machine monitor to maintain a separate shadow page table.

EPT technology incurs extra overheads in cross-ISA DBT. Same-ISA virtualization can switch on EPT during the execution of a guest VM, and switched off EPT when the guest VM invokes a hypercall, thus CPU mode only switches on and off EPT when entering and exiting the VMM. A cross-ISA DBT, however, needs to switch CPU mode each time the guest accesses a virtual address. The reason is that the guest is not in the same address space with the DBT while EPT can only be activated in VM mode, and therefore, the emulator needs to switch to VM mode to perform the address translation with EPT and then exit the VM mode. Such frequent CPU mode switching incurs very high overhead in cross-ISA DBT. Another issue of EPT is that currently it is only available on x86 platform, thus making EPT not a general solution for address translation in cross-ISA DBT.

3. Design of Hardware-Assisted Memory Address Translation for Cross-ISA System Emulation

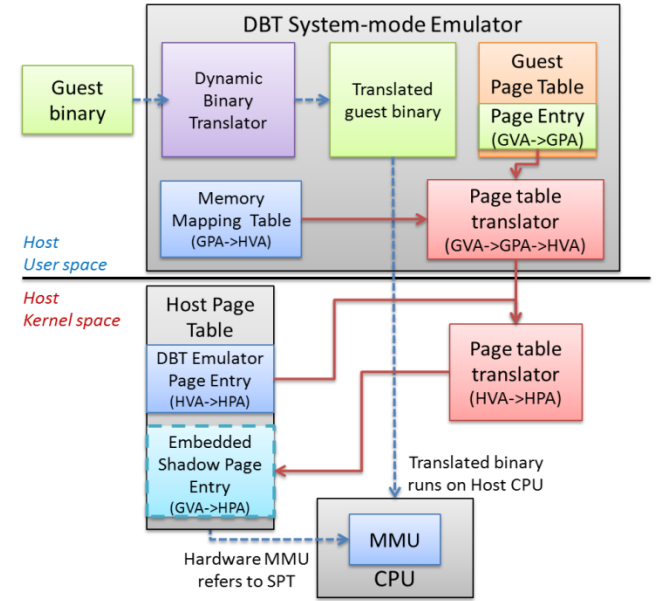


Figure 3-0. System architecture of hardware assisted memory address translation for cross-ISA system emulation

To eliminate the aforementioned bottlenecks, we propose an “*embedded shadow page table*” (ESPT). When a DBT emulates a guest memory access instruction, it creates an entry in shadow page table for mapping guest address to host address. To avoid frequent table pointer switching as described in section 2.2, we embed the shadow page table entries in the host page table of the cross-ISA DBT. As a result, the host page table now has two kinds of entries: (1) the original address mapping entries of the host DBT emu-

lator, and (2) the embedded guest to host address mapping entries. Since we embed the shadow page entries into the host page table, the hardware MMU can use these “*embedded*” shadow page entries in the host page table to translate memory address transparently. Therefore, our ESPT approach does not need to switch between host page table and shadow page table, and can achieve the same efficiency as in a native environment.

Figure 3-0 illustrates the system architecture of the proposed hardware assisted memory address translation. The page table translator merges the multiple-level address translation into one level of guest-to-host address translation. After the merging, if the guest binary issues a memory operation the MMU of the host CPU looks up the embedded shadow page table to translate the guest address to a host address directly and transparently. To achieve this one level translation, the page table translator first walks the guest page table to find the corresponding guest physical address, then it searches the memory mapping table in the DBT emulator to find the corresponding host virtual address for the guest physical address. Finally, the page table translator walks the host page table to find the corresponding host physical address for the host virtual address.

There are two major challenges in designing such a framework. The first challenge is to find a memory region in the DBT address space that does not conflict with the DBT emulator itself. In system-mode DBT emulation, all guest instructions are translated and executed on a host CPU within the same address space as the DBT emulator. We need to find an efficient way that can adapt DBT to use hardware MMU to speed up cross-ISA emulation.

The second challenge is to reduce the overhead of creating and maintaining the embedded shadow page table. Recall that in our approach, the hardware MMU uses embedded shadow page entries to translate guest virtual address to host physical address directly. We can create the embedded shadow page table entries for the entire guest page table at the beginning of emulation, however, such an approach will incur a lot of inefficiency. The reasons are: (1) When an entry in the shadow page table is created for a guest page table entry, synchronization between guest and shadow page tables is required to ensure the correctness of the mapping, (2) Not all the entries of the guest page table will be accessed during the emulation. If we create shadow page entries for the entire guest page table at the beginning of emulation, it will incur excessive synchronization overhead. How to dynamically create embedded shadow page table entries to avoid unnecessary synchronization overhead is a challenging issue. In the following subsections we present our design approaches to tackle these issues.

3.1 Memory Allocation and Design of Shadow Page Table

A cross-ISA DBT runs as a process in the user space of the host virtual memory. As described in Figure 2-1, the host OS creates a page table for the DBT emulator process. We call the page table “host page table” (part (A) in Figure 3-1). The guest VM (including guest OS and the applications running on it), emulated by the DBT, has its own “guest page table” (part (B) in Figure 3-1). The DBT maintains an internal table (part (C) in Figure 3-1) to manage the memory allocation for the guest VM.

Before creating an embedded shadow page table, we must determine its location so that it will not interfere with the virtual memory space of the guest virtual machine. A guest operating system running on a 32-bit architecture, such as x86_32 and ARM_32, will only use the address space below 4GB, so the address above 4GB is free and can be used for other system components. To free the address space below 4GB, we can modify the linker script of the DBT so that it will place itself and its components above 4GB in the host virtual memory during runtime. We then use the free space of host page table entries below 4GB to store the embedded shadow page table. When our system emulates a 32-bit guest memory operation, the page table translator will find an unused page table entry below 4GB in the host page table to create a guest virtual address to host physical address mapping. We call such page table entry an embedded shadow page entry. Because the address space below 4GB is free, it is safe to map any 32-bit virtual address here.

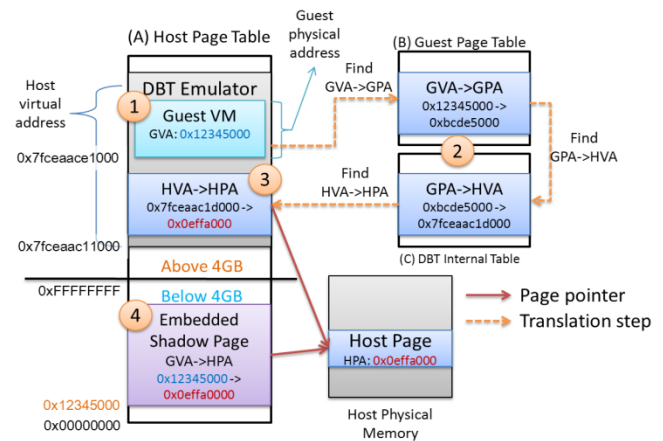


Figure 3-1. Memory allocation and design strategy for embedded shadow page table

In this paper, we focus on 32-bit guest and 64-bit host. Memory allocation for 64 bit guest to 64 bit host translation is more complicated but still possible. The main issue for 64-bit on 64-bit is how to find free space on host address space. We think this is still possible because 64-bit is a very large space and most OSes and applications do not use all

the space. For example, Linux only uses virtual address from 0x7f00'0000'0000 and above. Thus, it is still possible to find a free space to place the ESPT. We leave it to our future work. We do not consider 64 bit guest to 32 bit host translation because such kind of translation is rarely needed.

Figure 3-1 illustrates our shadow page table design. We assume that we load the DBT at host virtual address 0x7ceaac11000, and the DBT uses virtual address 0xfceaace1000 as the starting point of the guest physical address space. When a guest operating system accesses a virtual address 0x12345000 (indicated in (1)), if the address does not exist in shadow page table, the host page fault exception triggers our page table translation mechanism. Next, our framework traverses the guest page table to find the corresponding guest physical address, and then looks up the internal table of the DBT to find the corresponding host virtual address (indicated in (2)). Next it searches the host page table to find the corresponding host physical address (indicated in (3)). Then it creates a shadow page table entry for that guest virtual address and embeds the entry into the host page table of the DBT (indicated in (4)). This entry has the guest virtual address 0x12345000 and points to the same corresponding host physical address.

3.2 Incorporating with DBT emulator

In this section, we discuss how to make this scheme work with a DBT system mode emulator. Here we use QEMU as an example. We need to modify the QEMU tiny code generator (TCG) to use the hardware to translate the guest address. QEMU uses a 'qemu_ld_helper' helper function to translate memory address in software. Figure 3-2 shows an example of translated host code from a guest memory operation 'mov 0x4(%ebp), %eax'. This guest memory instruction loads the memory address pointed by %ebp plus an offset 0x4 into %eax. QEMU maps the guest register %ebp as 0x20(%r14) in host memory. QEMU generates the following codes to emulate this instruction. (1) load guest register %ebp (0x20(%r14) in host binary) to host register %eax, (2) add the offset 0x4 to %eax. Now the guest virtual address is in host %eax, (3) use SoftMMU to find the host virtual address, and (4) load the data in the host virtual address.

In Figure 3-2, the left hand side is the original QEMU soft-MMU approach that employs two alternatives---soft-TLB (fast_path) and soft-MMU (slow_path). The fast_path is a series of instructions to search the table by a hash function, and the slow_path is a guest page table walking procedure. On the right hand side is our approach that generates only one 'mov' instruction to access the guest memory. Our approach uses 10 to 20 less instructions than the soft-TLB and soft-MMU approach in QEMU. In our approach QEMU is a process on the host so it can use the host page table, which already contains the shadow page table. As a

result we can run guest instruction and access the intended guest memory address without switching among multiple page tables.

Guest instruction: mov 0x4(%ebp), %eax
 → Load %eax from the address in %ebp + offset 0x4
 → Guest %ebp is mapped in host 0x20(%r14)

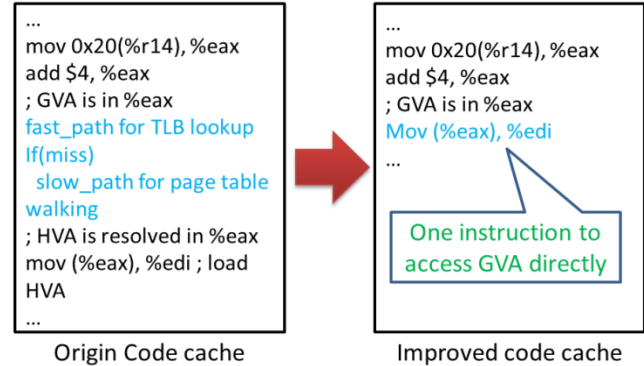


Figure 3-2. Improved QEMU code cache with hardware-assisted address translation

There are two types of branch instructions in DBT -- direct and indirect branch. Direct branch can be handled by ESPT efficiently. For indirect branch, we use Indirect Branch Translation Cache (IBTC) [14] to cache the lookup result so that we do not need to look up the entire table every time we need to determine a destination address.

3.3 Reducing Overhead of Creating Shadow Page Table

The shadow page table needs to be created before guest OS can access it. Usually a guest page table contains many page entries, therefore it is inefficient to translate all entries at once at the beginning because not all entries will be used. Moreover, synchronization between shadow and guest page tables also incurs heavy overhead because the guest OS modifies its page table very frequently, e.g., from lazy page allocation. Lazy allocation will not allocate the free memory page immediately when a guest application calls 'malloc' of glibc library to allocate memory space; instead it only allocates space when the application accesses the allocated address for the first time. As a result the lazy allocation will update the page table numerous times, and causes significant overheads.

Interception based methodology for reducing synchronization overhead. Our approach only creates pages that are needed or accessed by the guest operating system in order to reduce the overhead in creating the shadow page table. To synchronize entries in shadow page table, our framework intercepts the 'invalid TLB' instruction issued by the guest operating system when a guest application maps, unmmaps or changes an existing mapping. By intercepting

invalid TLB instructions, our framework only needs to update the modified page table without constant monitoring modifications made to the guest page table. This design significantly reduces overheads in tracing guest memory operations.

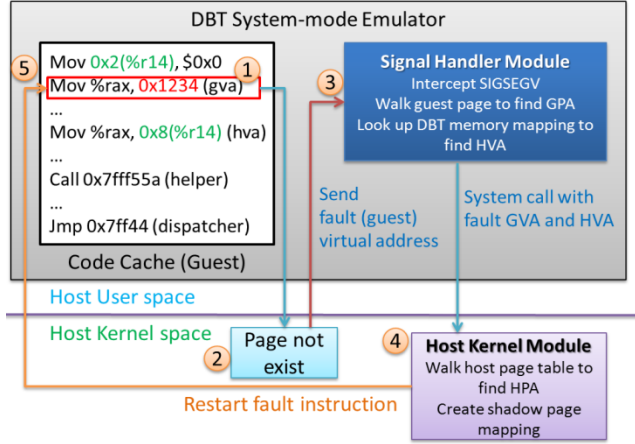


Figure 3-3. Signal notification approach for creating embedded shadow page table on demand

Signal notification mechanism for dynamic recreation of ESPT. We propose a signal notification mechanism that reduces the size and overheads in creating embedded shadow page table by only creating the shadow page table entries on demand. The flow of signal notification is illustrated in Figure 3-3. We add a host kernel module to help manipulate the host page table without modifying the host kernel. The host kernel provides the function to create memory alias and security checking on the memory address to make sure it will not touch the DBT emulator or other system components. When a guest virtual address is accessed for the first time (labelled as step 1), the host kernel raises a SIGSEGV signal (step 2) because right now this guest virtual address does not yet have a corresponding entry in the host page table. The signal event contains the faulty memory address that causes the signal. We also add a signal handler into the DBT to intercepts this SIGSEGV signal (step 3) and perform a series of address translation: a) walk the guest page table to retrieve corresponding guest physical address, b) look up the internal memory mapping table of DBT to get the corresponding host virtual address, c) pass the guest virtual address and the host virtual address to our kernel module by a system call, d) the kernel module walks the host page table to finds the corresponding host physical address of the host virtual address, and adds an entry from the guest virtual address to the host physical address in the embedded shadow page table (step 4). After adding the embedded shadow page table entry, the host kernel restarts the instruction (step 5). Now the 'mov' instruction will correctly read or write the requested guest virtual address.

3.4 Page fault handling

Page fault is an important system design issue in page table manipulation. There are two types of page faults in our approach -- shadow page fault and guest page fault. A shadow page fault occurs when the requested guest virtual address is not in the shadow page table. A guest page fault occurs when the address does not exist in the guest page table. Figure 3-4 illustrates our approach of two-level page fault handling and the Left hand side is a translation block in the DBT code cache. Our approach adds a 'mov' and a 'jmp' instruction before executing DBT SoftMMU. Our approach tries the shadow page first while trying to access a guest virtual address. If the entry for the guest virtual address does exist in the shadow page table, we translated it into host physical address by hardware MMU directly. If the entry for the guest virtual address does not exist in the shadow page table nor the guest page table, we will disable hardware MMU instruction by replacing the 'jmp' with a 'nop' so the DBT emulator will use soft-MMU to fill the guest page table.

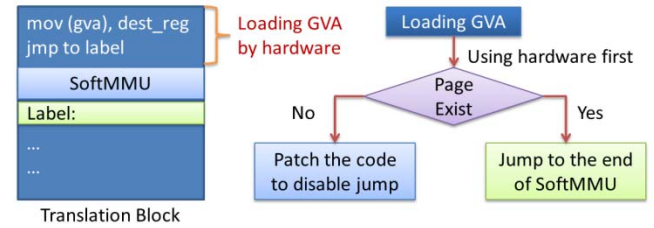


Figure 3-4. Page fault handling

3.5 ESPTs for Multi-Process Emulation

In this subsection, we describe how we handle multiple guest processes by using one host page table. In system mode emulation, there are usually more than one processes running in guest VM. The guest OS needs to do context-switching between each process periodically to enable multiple processes to share a single CPU. This is also an essential feature of a modern multitasking operating system.

We create one shadow page table per guest process to eliminate unnecessary re-creation of shadow page table. The x86_64 platform has a four-level page table, and in each level a page table maintains 512 tables of the next level. The top level page table is called page global description table (PGD). The tables in the next three levels are page upper description table (PUD), page middle description (PMD) table, and page table (PT) respectively. Each table in the first three levels has 512 entries and each entry points to a table in the next level. Each entry of a page description table in the last level points to a 4096-bytes physical memory page. As a result four entries in the second level table can address 4G memory ($4096 \times 512 \times 512 \times 4$),

which is sufficient to support a 32-bit guest operating system, as shown in Figure 3-5.

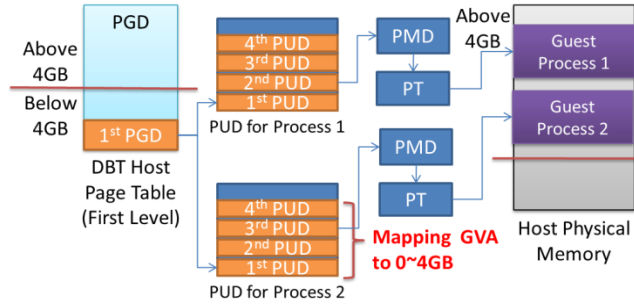


Figure 3-5. Multi-process emulation uses the first entry in the top level page table as the shadow page table pointer

Figure 3-5 illustrates the design of our embedded shadow page tables for multi-process support using only one host page table. We create four second level page table entries for each guest process and use the first entry in the top level table as the shadow page table pointer. When the guest OS switches context between two processes, our framework changes the first entry in the top level table from pointing to the four second level entries of the leaving process to the four second level entries of the incoming process. Our framework also set the write-protection on these switched-out shadow page table to keep it synchronized with guest page table. This design can reduce the time needed for recreating the shadow page table.

Our framework needs to identify the current running process in the guest operating system in order to achieve multi-process emulation. It is more precise to use the guest process id to identify the running process than to use the guest page table pointer, because guest operating system may reuse the same page table pointer for new processes without issuing page table modification instruction. The process id is an architecture information located in control registers on an ARM platform, but is not available on x86 platform. Thus we slightly modify the guest operating system kernel to retrieve the current running process id on an x86 platform.

4. Experimental Results

We implement our hardware-assisted ESPT based memory virtualization framework on QEMU, a popular retargetable cross-ISA dynamic binary translator. We evaluate the effectiveness of our framework on Intel x5550 machine with 2.67GHz CPU, 12G RAM, and an x86_64 Gentoo Linux operating system as QEMU host. We use Gentoo Linux 201308 as our guest OS and runs benchmarks in the guest VM. All the benchmarks we use can run correctly in such multi-tasking environment. Two benchmark suites are used as the guest applications in our experiments: SPEC CINT2006 and NBench. SPEC CINT2006 is

an industry-standardized, CPU-intensive benchmark suite, stressing system's processor and memory subsystem. The reason we chose the SPEC integer benchmark suite is that QEMU v0.15 and up uses integer instructions to simulation floating point instructions for portability. Such software simulation approach causes significant slowdown on the SPEC floating point benchmarks.

NBench is a synthetic computing benchmark program and is intended for measuring a computer's CPU, FPU, and memory system speed. In our experiments, we emulate ARM and x86_32 platforms with Gentoo Linux OS on an x86_64 machine. In Section 4.1, we first describe the implementation work needed to make the ARM MMU system work on an x86 machine.

4.1 MMU on ARM Platform

ARM processor provides a set of CP15 registers to control the behavior of the processor, including cache and MMU. CP15:C1 register decides whether MMU is enabled or not. CP15:C2 register points to the translation table base which contains the mappings of guest virtual address to guest physical address. CP15:C8 register provides the TLB-related operations, such as 'TLBIMVAA', which invalidates a TLB entry of the virtual address, and 'TLBIALL', which invalidates all TLB entries. To do cross-ISA address translation, we first need to check if the MMU is enabled (C1). If it is enabled, we can walk the ARM page table starting from the translation table base (C2).

4.2 ARM 32bit to x86 64bit DBT

In this experiment, we emulate an ARM platform with Linux OS on an x86 machine. Figure 4-2(a) shows the performance of the CINT2006 benchmarks with train input size in ARM-to-x86 emulation. The reason that we use train instead of reference input size is the memory limitation. The ARM platform emulated by QEMU can only provide maximum 1GB memory for guest OS while some benchmarks require more than 1GB memory during execution. The performance will be affected by memory swapping, thus we use train input size for ARM-to-X86 emulation. The x-axis is the benchmarks and the y-axis is the execution time in seconds. The 'soft' legend is the execution time with QEMU Soft-MMU, and the 'hard' legend is the execution time with our hardware-assisted MMU. Figure 4-2(b) shows the speedup. On average, our approach achieves 1.51X speedup. We can also see that among the twelve benchmarks, 429.mcf has the largest speedup (2.95X), while 445.gobmk has the smallest (1.20X). We will explain the reasons in Section 4.4.

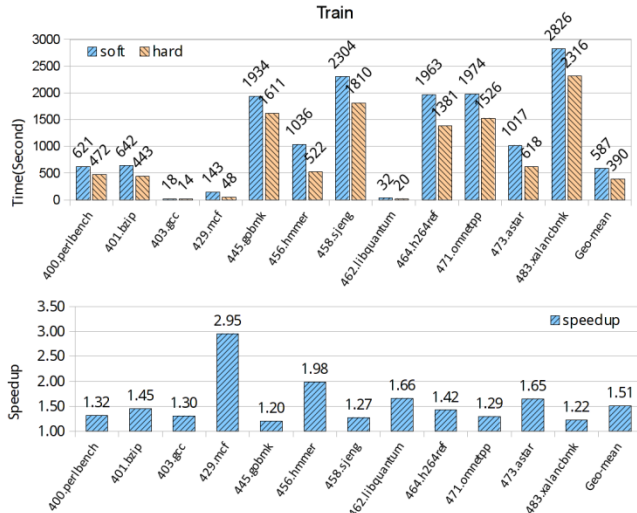


Figure 4-2. (a) Execution time and (b) Speedup by hardware MMU against software MMU for the SPEC CINT2006 benchmarks in ARM-to-x86 emulation (with train input).

We also compare the effectiveness of our hardware assisted memory virtualization on short running tasks versus long running tasks. We use SPEC CINT2006 with test input (short running) and reference input (long running) as the basis for comparison. The difference between short and long tasks is that longer running code will have more opportunities to exploit the ESPT entries once they are created. Comparing Figure 4-2(b) and Figure 4-2-2, we can see that the reference input (long running) results in larger speedup factors (1.51X on average) than the test input (short running), 1.37X on average. The reason is that, with the increase in execution time, the reuse ratio of the ESPT also increases, hence resulting in better performance.

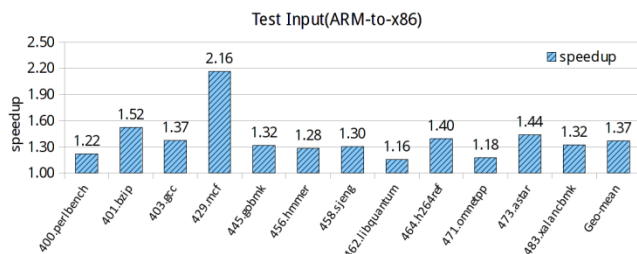


Figure 4-2-2. Speedup by hardware MMU against software MMU for the SPEC CINT2006 benchmarks in ARM-to-x86 emulation (with test input)

Figure 4-2-3 shows the experiment result on NBench. On average, our approach achieves 1.22X speedup against the original QEMU. Among the 7 benchmarks, String Sort achieves the most significant speedup (1.64x) because it is a memory-intensive program which performs a large number of memory read and compare. BitField has the lowest speedup (1.06x) because it has few memory operations. Its

computation mainly involves bit manipulation functions, such as: AND, OR, and XOR.

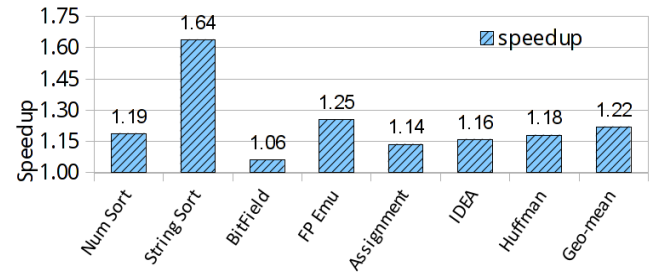


Figure 4-2-3. Speedup by hardware MMU against software MMU for the NBench benchmarks in ARM-to-x86 emulation.

The performance improvement for NBench is not as good as that of SPEC CINT2006. The main reason is that most of the NBench benchmarks do not have many memory operations as most of the SPEC CINT2006 benchmarks do.

4.3 x86 32bit to x86 64bit DBT

x86 32bit is one of the most popular computing platforms. Many applications (and operating systems) were developed on x86 32bit machines. With the increasing popularity of x86 64bit machines in recent years, there will be increasing need for emulating legacy operating systems and applications developed for x86 32bit on the newer x86 64bit platforms. Hence, we also conduct experiments on x86_32 to x86_64 DBT.

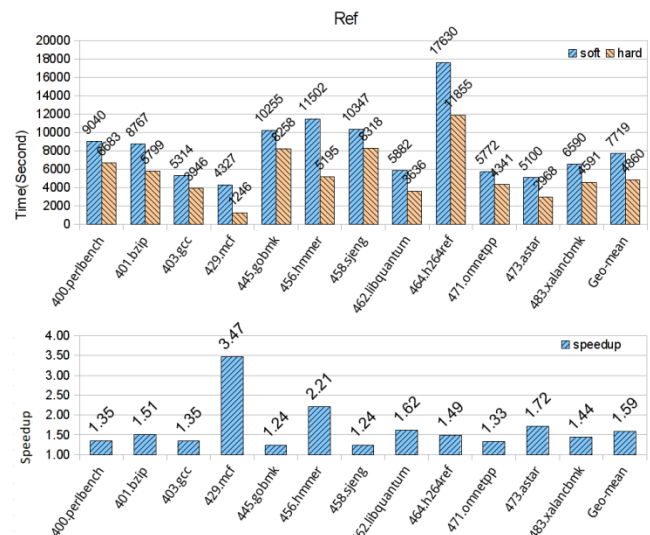


Figure 4-3. Speedup by hardware MMU against soft MMU on SPEC CINT2006 benchmarks for x86_32 to x86_64 DBT (with reference input)

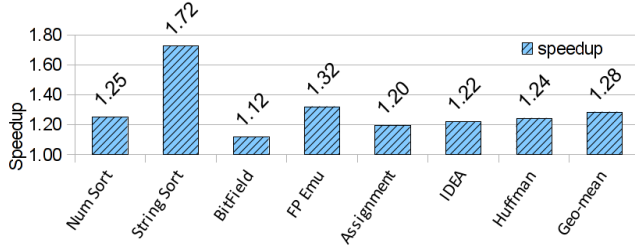


Figure 4-3-2. Speedup by hardware MMU against soft MMU on NBench benchmarks for x86_32 to x86_64 DBT

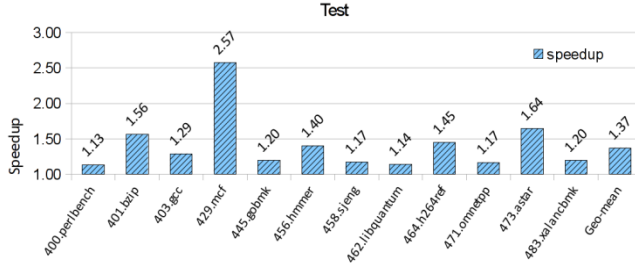


Figure 4-3-3. Speedup by hardware MMU against soft MMU on SPEC CINT2006 benchmarks for x86_32 to x86_64 DBT (with test input)

Figure 4-3 and 4-3-2 show the execution time and speedup factors of the SPEC CINT2006 and the NBench benchmarks respectively. The speedup factors on SPEC CINT2006 are quite consistent with those for ARM-to-x86_64 emulation, with an average speedup of 1.59x compared to SoftMMU. For NBench, the average speedup is 1.28x. String Sort achieves the most significant speedup (1.72x) while BitField has the lowest speedup (1.12x).

By comparing Figure 4-3 and Figure 4-3-3, we can see that short running tasks versus long running tasks have similar effect on the speedup factors of CINT2006. That is, long running tasks (with reference input) result in higher ESPT reuse ratios and hence larger performance improvement.

In the next subsections, we examine the effect of hardware assisted address translation on reduction of memory instructions and TLB miss rate.

4.4 Effect on Memory instructions Reduced

From processor microarchitecture point of view, memory operation is one of the slowest instructions, thus processor manufacturers add many mechanisms to reduce the frequency of memory accessing, such as multiple level memory caching and translation lookaside buffer. As mentioned in previous section, our framework can reduce the level of memory translation from 3 steps to just 1 step. Even in the case of QEMU Soft-TLB hit, our framework can also reduce the number of memory instructions from 3 to 1, because Soft-TLB needs (1) calculate guest virtual address (2) read host virtual address from hash table (3)

read the data in the host virtual address (and then the hardware MMU will translate the virtual address to the host physical address transparently). Figure 4-4 shows the percentage of memory instructions eliminated by our framework. In most benchmarks, the speedup is proportional to the reduction percentage of memory instructions.

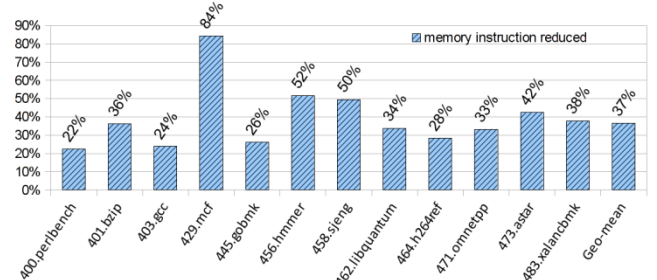


Figure 4-4. Reduction percentage of memory instructions by hardware-assisted address translation

4.5 Effect on TLB miss rate

429.mcf is the one that achieves the highest performance improvement. The reason is described as follows. Recall that in Figure 1-1, 429.mcf takes about 40% time in soft-MMU, and other benchmarks, such as 401.bzip and 473.astar, also take about 40% time in soft-MMU. Why is 429.mcf the only one that achieves 3.47X speedup while 401.bzip and 473.astar only has 1.51X and 1.72X speedup respectively? Hardware MMU can reduce memory operation instructions, however, in Figure 4-5, all benchmarks contain about 26% memory accessing instructions, 429.mcf is not the one that has the highest number of memory access instructions. Therefore, reduction in the number of memory operations is not the main reason that makes 429.mcf achieve such high performance improvement.

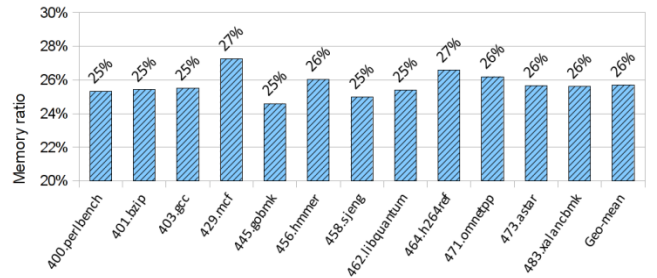


Figure 4-5. Percentage of memory access instructions in SoftMMU mode

Another possible reason would be SoftTLB in QEMU. A TLB is a cache that memory management hardware uses to improve virtual address translation speed. In order to speed up SoftMMU, QEMU also implements a TLB-like caching mechanism that records the guest virtual address to host virtual address mapping. Figure 4-5-2 shows TLB

miss rate of SoftTLB(left bar) compared with our framework(right bar). 429.mcf causes 10%TLB miss in SoftTLB while others only 0.1% ~ 3%. The reason is that QEMU's SoftTLB is implemented using hash table that uses the last 8 bits of guest virtual address as the index, and therefore can hold up to 256 entries. The hash design has the best performance when finding an entry. However, this simple design will suffer from hash conflicts. For example, these two virtual addresses, 0x1234aa12 and 0x1234bb12, will compete for the same hash entry and causes SoftTLB invalidation and SoftTLB miss. For HardTLB, current processor provides larger and multiple level TLB. For instances, Intel's Nehalem microarchitecture has a L1 DTLB with 64 entries and a L2 TLB with 512 entries, both four-way associative. With the hardware support, our framework can decrease TLB miss rate of 429.mcf from 10% to 1%, and thus obtain 3.47 times speedup.

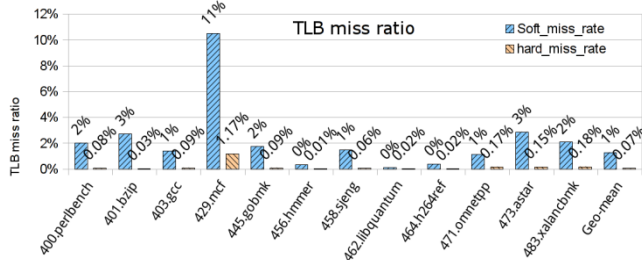


Figure 4-5-2. TLB miss rate of QEMU SoftTLB compared with Hardware-supported TLB

We also enlarged QEMU SoftTLB. We found that QEMU reaches its peak performance when the SoftTLB is increased to 2^{12} . However, even with peak performance of QEMU's SoftTLB hit, our approach still achieves 1.5X speedup against QEMU.

From the results presented in Section 4.4 and Section 4.5, we can see that hardware supported memory virtualization can not only reduce the memory address translation overhead but also reduce the TLB miss rate, because hardware provides more effective cache replacement mechanism.

5. Related work

There are generally two types of binary translators: (1) process-level binary translators and (2) system-level binary translators. Process-level binary translation emulates the functionality of the application binary interface (ABI) of the guest application. On the other hand, the system-level binary translators emulate the functionality of the entire guest ISA interface, including privileged instructions. We could further classify binary translators into four categories according to the relationship between the platform ISA and the operating system. Table 5-1 categorizes the binary translators into four possible scenarios.

Table 5-1. The classification of binary translators.

ISA / OS	Same ISA	Cross ISA
Same OS	DynamoRIO[4,15], Strata[5], Embra[2], Pin[3]	FX!32[6], memTrace[12], StarDBT[10], Valgrind[18]
Cross OS	VMware [7], Xen [8], KVM[9], VirtualBox [11].	QEMU[13], Quick-Transit [16]

Binary translators in the same-ISA-same-OS category are usually for improving performance, security, and binary instrumentation. Same-ISA-cross-OS virtualization works are usually for server virtualization, which is extremely important for cloud computing. Related works in the cross-ISA-same-OS are most for application migration, optimization and instrumentation. They are all bound for a specific guest-ISA/host-ISA combination. MemTrace [12] is a memory tracing framework for 32-bit applications. It uses the wider address space available on 64-bit machine to isolate the tracer from 32-bit guest program. StarDBT [10] is an optimizer which uses the new registers and instructions provided on 64-bit machine to optimize the legacy 32-bit code. Valgrind [18] is a framework for dynamic binary instrumentation. Users can use it to build dynamic analysis tools. In the Cross-ISA-cross-OS category, QuickTransit [16] supports multiple source and target ISAs, but also allows different source and target OS's. QEMU [13] is a retargetable binary translator. It currently supports six guest-ISAs (ARM, CRIS, m68k, MIPS, SPARC, IA32) and 3 host-ISAs (IA32, x86_64, PowerPC) with several operating systems.

Since memory virtualization usually relies on the underlying host architecture, we will focus on same-ISA and cross-ISA memory virtualization.

In same-ISA, memory virtualization can be achieved by software-based shadow page table [20,22] and hardware-supported Extended Page Table(EPT) technology [9]. The implementation of shadow page table used by VMware optimizes caching of guest virtual address translations so that expensive hardware faults resulting from memory access are minimized. Dynamic binary translation is also used to keep the cost of MMU-related operations low. With shadow page tables, the virtual machine's linear addresses can be mapped directly to the physical machine addresses, and therefore, accesses to memory can be performed at native hardware speed.

In cross-ISA system mode emulation, the guest page table structure is different from host, thus hardware cannot be used to translate guest address directly. Some cross-ISA emulators provide a software-based MMU[13] to translate guest address. QEMU is one of the cross-ISA system-mode

emulators that use Soft-MMU for memory virtualization. However, as evidenced in this paper, Soft-MMU incurs high overhead and could become the performance bottleneck of cross-ISA emulators.

6. Conclusion

We have proposed an efficient memory virtualization scheme, “*embedded shadow page table*” (ESPTs), for cross-ISA system-level DBT that uses shadow page tables and host hardware MMU to reduce address translation overheads. By embedding the shadow page entries into the host page table of the DBT, we can avoid frequent switching between host and shadow page table during address translation. We have also proposed two optimizations for reducing ESPT overhead and hence increasing performance gain. We adopt an interception-based methodology to reduce the overhead of synchronizing guest page table and embedded shadow page table. We have also devised a signal notification mechanism which creates the shadow page table entries on demand. These optimizations have successfully reduced the overhead of creating embedded shadow page table. We also take the advantage of wider address space available on 64-bit CPUs embed our shadow page table and isolate the guest OS from DBT emulator.

Our experiment results demonstrate an average speedup of 1.51 times and 1.59 times on SPEC CINT2006 benchmarks for ARM-to-x86 and IA32-to-x86_64 system-mode emulation respectively, and an average speedup of 1.22 times and 1.28 times on NBench benchmarks for ARM-to-x86 and IA32-to-x86_64 system-mode emulation respectively, when compared with the standard softMMU approach in QEMU. We also show that our approach reduces the number of memory instructions by 37% on average and reduces TLB miss rate by more than 10 times compared with the original QEMU.

Acknowledgments

We would like to express our gratitude to Dr. Donald Porter for his very helpful comments and suggestions to improve this paper. This work is supported in part by National Science Council of Taiwan under grant number NSC102-2221-E-001-034-MY3.

References

- [1] James Smith and Ravi Nair, “Virtual Machines: Versatile Platforms for Systems and Processes”, Elsevier, 2005, ISBN: 1-55860-910-5
- [2] E. Witchel and M. Rosenblum, “Embora: Fast and Flexible Machine Simulation”, Proceedings of the ACM SIGMETRICS international conference on Measurement and modeling of computer systems, 1996
- [3] Luk, C.-K., Cohn, R., Muth, R., Patil, H., Klauser, A., Lowney, G., and Wallace, S., Reddi, V. J., Hazelwood, K., “Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation”, Programming languages design and implementation, June 2005.
- [4] N. Nethercote. Dynamic Binary Analysis and Instrumentation. A dissertation submitted for the degree of doctor of philosophy, University of Cambridge, November 2004.
- [5] K. Scott and J. Davidson. Strata: A Software Dynamic Translation Infrastructure. In IEEE Workshop on Binary Translation (2001).
- [6] A. Chernoff and Ray Hookway, “DIGITAL FX!32: Running 32-Bit x86 Applications on Alpha NT”, the Proc. of the USENIX Windows NT Workshop, August 1997
- [7] VMware. <http://www.vmware.com>.
- [8] Xen. <http://www.xen.org>.
- [9] Sheng Yang, “Extending KVM with new Intel® Virtualization technology”, Intel Open Source Technology Center, KVM Forum 2008.
- [10] WANG, C., HU, S., KIM, H.-S., NAIR, S., BRETERNITZ, M., YING, Z., and WU, Y. “Stardbt: An efficient multi-platform dynamic binary translation system”, In Advances in Computer Systems Architecture, vol. 4697. 2007
- [11] Virtualbox. <http://www.virtualbox.org/>.
- [12] Payer, M., Kravina, E., and Gross T. R., “Lightweight Memory Tracing”. In USENIX ATC, 2013
- [13] Bellard, F., “QEMU, a Fast and Portable Dynamic Translator,” in USENIX ATC 2005
- [14] K. Scott, N. Kumar, B. R. Childers, J.W. Davidson, and M. L. Soffa, “Overhead reduction techniques for software dynamic translation,” in Proc. IPDPS, 2004, pp. 200–207.
- [15] D. Bruening, T. Garnett, S. Amarasinghe, “DynamoRIO: An Infrastructure for Adaptive Dynamic Optimization”, Proceedings of the international symposium on code generation and optimization, 2003.
- [16] Transitive corporation. quicktransit software. <http://www.transitive.com/>.
- [17] Xiao Guangrong, “KVM MMU Virtualization”, Linux Foundation Events, 2011.
- [18] N. Nethercote and J. Seward, “Valgrind: A Framework for Heavyweight dynamic Binary Instrumentation”, ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2007), June 2007
- [19] E. Bugnion, S. Devine, M. Rosenblum, J. Sugerman and E. Y. Wang, “Bringing Virtualization to the x86 Architecture with the Original VMware Workstation”, TOCS, volume 30 issue 4, Nov 2012
- [20] K. Adams and O. Agesen, “A Comparison of Software and Hardware Techniques for x86 Virtualization”, ASPLOS’06, San Jose, California, USA, October 21–25, 2006.